SOURCE CODE

```
`timescale 1ns/1ps
module topm (
   input       clk,
   input       reset,
   input  [3:0] req,
   output [3:0] gnt
);
   epla_arbiter dut (
      .clk(clk),
      .reset(reset),
      .req(req),
      .gnt(gnt)
   );
endmodule

module epla_arbiter (
   input       clk,
   input       reset,
   input  [3:0] req,
   output reg [3:0] gnt
);

   reg [1:0] dyn_lock;
   reg [1:0] lock_cnt;
   wire [2:0] req_count = req[0] + req[1] + req[2] + req[3];

   always @(*) begin
      case (req_count)
         3'd0, 3'd1: dyn_lock = 1;
         3'd2:       dyn_lock = 2;
```

```
        default:    dyn_lock = 3;
    endcase
end
wire g0_active = req[0] | req[1];
wire g1_active = req[2] | req[3];
reg token;
reg lock_active;
reg g0_ptr, g1_ptr;
reg first_cycle;
reg [3:0] next_gnt;
wire gnt_en = |next_gnt;
always @(posedge clk or posedge reset) begin
    if (reset) begin
        gnt        <= 4'b0000;
        token      <= 0;
        g0_ptr     <= 0;
        g1_ptr     <= 0;
        lock_cnt   <= 0;
        lock_active <= 0;
        first_cycle <= 1'b1;
    end else begin
        next_gnt <= 4'b0000;
        if (first_cycle && req[0]) begin
            next_gnt   <= 4'b0001;
            lock_active <= 1;
            lock_cnt   <= dyn_lock;
            first_cycle <= 0;
        end
        else if (lock_active) begin
            if (lock_cnt == 1) begin
                lock_active <= 0;
```

```
      if (token==0 && g1_active) token <= 1;
      else if (token==1 && g0_active) token <= 0;
    end else begin
      lock_cnt <= lock_cnt - 1;
    end
  end
  else begin
    if (token==0 && g0_active) begin
      if (req[0] && (g0_ptr==0 || req[1]==0)) begin
        next_gnt <= 4'b0001;
        g0_ptr   <= 1;
      end else if (req[1]) begin
        next_gnt <= 4'b0010;
        g0_ptr   <= 0;
      end
      lock_active <= 1;
      lock_cnt    <= dyn_lock;
    end
    else if (token==1 && g1_active) begin
      if (req[2] && (g1_ptr==0 || req[3]==0)) begin
        next_gnt <= 4'b0100;
        g1_ptr   <= 1;
      end else if (req[3]) begin
        next_gnt <= 4'b1000;
        g1_ptr   <= 0;
      end
      lock_active <= 1;
      lock_cnt    <= dyn_lock;
    end
    else begin
      if (token==0 && ~g0_active && g1_active) token <= 1;
```

```
            else if (token==1 && ~g1_active && g0_active) token <= 0;
        end
      end
      if (gnt_en)
        gnt <= next_gnt;
    end
  end
endmodule
```

TEST BENCH

```
`timescale 1ns/1ps
module tb_topm;

  reg clk, reset;
  reg [3:0] req;
  wire [3:0] gnt;

  // DUT
  topm uut (
    .clk(clk),
    .reset(reset),
    .req(req),
    .gnt(gnt)
  );

  // Clock generation
  initial clk = 0;
  always #5 clk = ~clk; // 10ns period

  // Initial reset
  initial begin
    $dumpfile("wave_manual.vcd");
```

```
        $dumpvars(0, tb_topm);

        reset = 1;
        req = 4'b0000;
        #20 reset = 0;

        // Keep simulation running for manual request changes
        // Use the simulator console to change 'req' manually
        forever #10; // simulation does not stop
    end

    // Display grants and internal signals
    always @(posedge clk) begin
        $display("T=%0t | req=%b | gnt=%b ",
                $time, req, gnt);
    end
endmodule
```

CONSTRAINTS FILE:

```
set_property CLOCK_DEDICATED_ROUTE FALSE [get_nets clk_IBUF]


##Switches


set_property -dict { PACKAGE_PIN J15   IOSTANDARD LVCMOS33 } [get_ports { reset }];
#IO_L24N_T3_RS0_15 Sch=sw[0]

set_property -dict { PACKAGE_PIN L16   IOSTANDARD LVCMOS33 } [get_ports { req[0] }];
#IO_L3N_T0_DQS_EMCCLK_14 Sch=sw[1]

set_property -dict { PACKAGE_PIN M13   IOSTANDARD LVCMOS33 } [get_ports { req[1] }];
#IO_L6N_T0_D08_VREF_14 Sch=sw[2]

set_property -dict { PACKAGE_PIN R15   IOSTANDARD LVCMOS33 } [get_ports { req[2] }];
#IO_L13N_T2_MRCC_14 Sch=sw[3]

set_property -dict { PACKAGE_PIN R17   IOSTANDARD LVCMOS33 } [get_ports { req[3] }];
#IO_L12N_T1_MRCC_14 Sch=sw[4]
```